



GATEWAY TECHNICAL

Unsupervised Learning



UNIVERSITY OF FLORIDA



1. Introduction to Unsupervised Learning

Unsupervised learning finds structure without labeled outcomes

Definition

Algorithms that discover patterns or relationships in data without target labels

Contrast with supervised learning

Contrast with supervised learning, which learns mappings from inputs → known outputs

Goal

Reveal latent structure — groups, patterns, or anomalies hidden in the data

Figure: Comparison of supervised learning (labeled outcomes) vs. unsupervised learning (unlabeled structure discovery)

The three pillars and goals of unsupervised learning



Clustering	Group similar observations to reveal structure
Dimensionality reduction	Simplify high-dimensional data for analysis or visualization
Anomaly detection	Identify rare or unexpected events that deviate from the norm

Together, these form the **core toolkit** for exploring and understanding unlabeled data.

When to use unsupervised methods



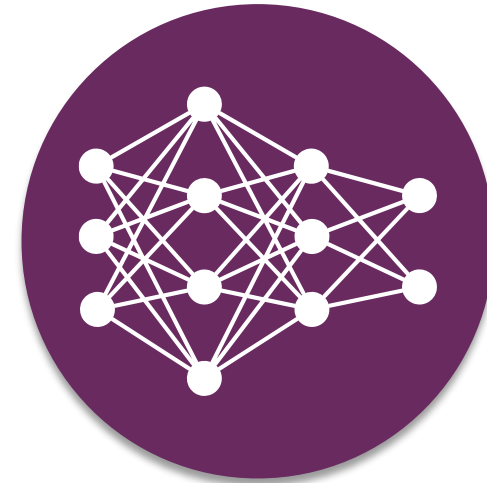
Exploration

Discover hidden relationships
before labels exist



Segmentation

Identify groups; Risk
stratification; Incident
typologies



Novelty detection

Detect emerging risks or rare
behaviors

From “nice-to-have” insight to mission-critical capability

- 
- **Segmentation** supports smarter marketing and personalization.
 - **Anomaly detection** safeguards against fraud, failure, or loss.
 - **Takeaway:** Unsupervised methods range from exploratory insight to **critical operational tools**.

Figure: Split visualization — Left: marketing segmentation; right: anomaly detection alert



2. Clustering: K-Means

K-Means assigns each point to the nearest centroid

Algorithm Steps

- 1 Randomly initialize k centroids.
- 2 Assign each point to the nearest centroid (Euclidean distance).
- 3 Recompute each centroid as the mean of its cluster.
- 4 Repeat until cluster assignments stabilize.

Output

Each data point gets a **cluster label**.

Figure: 2D points colored by cluster assignment with centroids marked by Xs

Initialization and convergence affect results

Initialization

- Random starting centroids can yield poor local minima.
- `k-means++` improves starting positions by spreading centroids apart.

Convergence

Algorithm always converges but not always to the global optimum.

Choosing k

- Elbow method: plot within-cluster sum of squares (WCSS) vs. k.
- Identify the “elbow” where added clusters give diminishing returns.

Figure: Line chart of WCSS vs. k showing elbow point around optimal cluster count

Feature scaling ensures fair distance comparisons



Preprocessing

Standardize or normalize features before clustering.

Goal

Make all variables contribute equally to distance calculations.

Scaling

Without scaling, features with large ranges (e.g., income) dominate smaller-scale features (e.g., age).

K-Means has practical limitations

Scale sensitivity

- Features on large numeric scales dominate distances.
- **Example:** Income (in \$) vs. purchase count.

Shape limitation

- Assumes clusters are convex and spherical.
- Struggles with elongated or irregular clusters.

Fixed k

Must specify the number of clusters in advance

Hierarchical clustering builds a tree of merges without pre-specifying k

Agglomerative

Start with each point;
Iteratively **merge the closest** clusters

Dendrogram: merge height

Dissimilarity;
Cut the tree at a height to get k clusters

Linkage choices

Complete (max), average (mean), single (min)

Distance

Euclidean common;
Correlation distance groups similar *profiles*

Figure: Dendrogram with a horizontal cut;
clusters below the cut in distinct colors



3. Clustering: DBSCAN and Density-Based Methods

DBSCAN groups points by density, not distance to centroids

K-Means

K-Means struggles with **irregular shapes** and **outliers**.

DBSCANS

DBSCANS key advantage is **flexibility** in capturing **nonlinear** or uneven density clusters.

Figure: Two datasets: left—K-Means enforces circular clusters; right—DBSCAN adapts to curved and irregular shapes.

DBSCAN groups points by density, not distance to centroids

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise):
 - Groups points based on **density** — not distance to a fixed centroid.
 - Automatically identifies **noise points** that belong to no cluster.
- Excels where K-Means fails — clusters of arbitrary shape or variable size.

Figure: Two datasets: left—K-Means enforces circular clusters; right—DBSCAN adapts to curved and irregular shapes

Core ideas behind density-based clustering

Density threshold

A cluster is a dense group of points separated by sparse regions.

Core point

Has at least `min_samples` neighbors within distance `eps`

Reachability

Points within `eps` of a core point belong to its cluster

Noise points

Isolated points not reachable from any dense region

Figure: Illustration labeling core, border, and noise points in a 2D dataset.

DBSCAN vs. K-Means: Flexibility vs. constraint

Feature	K-Means	DBSCAN
Cluster shape	Spherical / convex	Arbitrary (nonlinear)
Number of clusters	Must specify k	Found automatically
Noise handling	None	Identifies outliers explicitly
Scalability	Fast, simple	Slower on large datasets
Scale sensitivity	High	Moderate (still requires normalization)

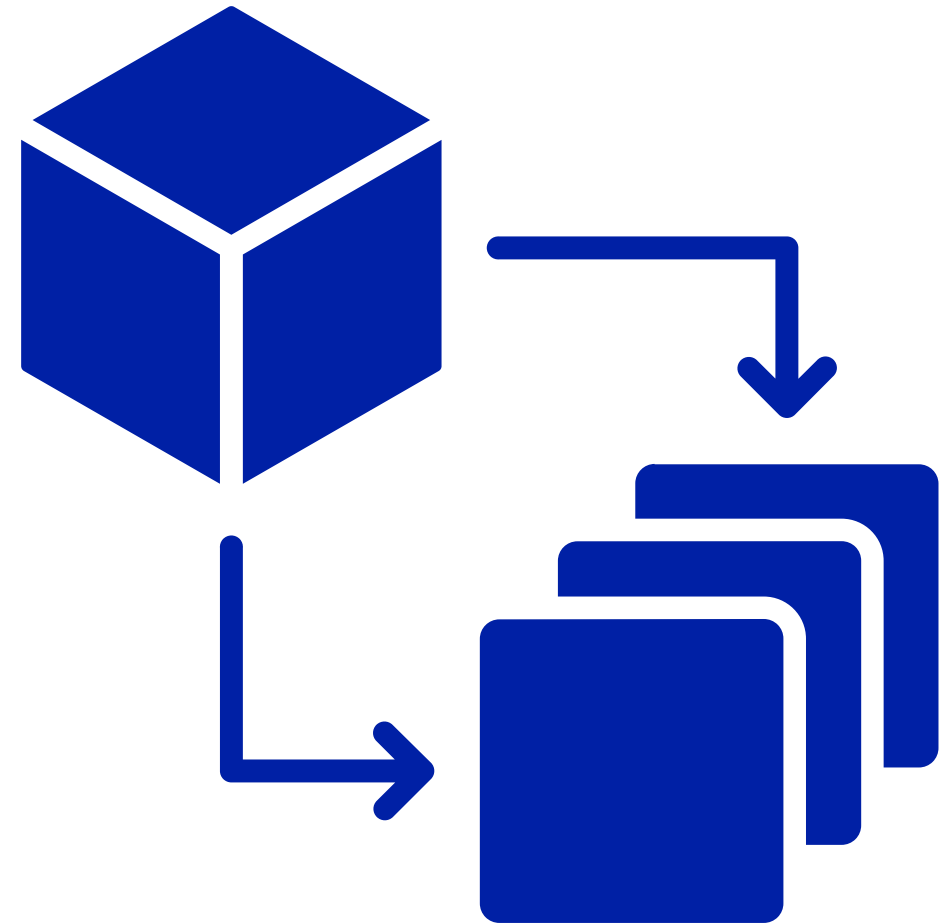


4. Dimensionality Reduction Techniques

Dimensionality reduction simplifies complex, high-dimensional data

- Real-world datasets often have **many correlated features**, making analysis and visualization difficult.
- Dimensionality reduction compresses features into fewer variables while preserving **essential structure and variance**.

Benefits
• Easier visualization and interpretation • Reduced noise and redundancy • Better performance for clustering and anomaly detection



Scaling features is essential before applying PCA

Why scaling matters

PCA depends on **variance**, and features with large numeric ranges dominate unscaled data.

Center and scale

Always **center and scale** variables before PCA unless all features share the same units.

Example

Without scaling, “Annual Income” (in dollars) outweighs “Age” (in years)

Equalizing

Scaling equalizes contribution across features → unbiased components.

PCA finds new orthogonal axes that capture maximum variance

- **Principal Component Analysis (PCA):** Finds new directions (principal components) that maximize variance.
- Each component is a **linear combination** of the original features.
- The **explained variance ratio** shows how much information each component retains.
- **Example:** The first two components may explain 80% of total variance → suitable for 2D visualization.
- **Limitation:** PCA is linear... it cannot model curved structures.

Figure: Scatterplot showing first two principal components as orthogonal axes;

Nonlinear methods capture complex structures PCA misses (out of scope for today, but included for completeness)

t-SNE (t-distributed Stochastic Neighbor Embedding)

- Preserves **local neighborhoods**, nearby points stay close after projection
- Excellent for visualizing clusters in image, text, or embedding data
- Computationally intensive and mainly for **exploration**

UMAP (Uniform Manifold Approximation and Projection)

- Balances **local** and **global** structure preservation
- Faster and more scalable than t-SNE
- Increasingly common for exploratory data visualization

Reduced-dimensional spaces often reveal anomalies

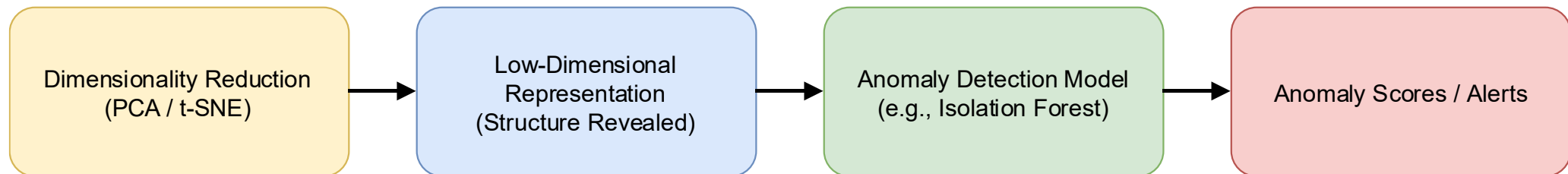
- **Anomalies often appear isolated** in reduced-dimensional projections.
- **PCA-based anomaly detection:**
 - Reconstruct input from principal components.
 - Large reconstruction error → potential anomaly.
- Dimensionality reduction can serve as both **preprocessing** and a **direct detection** tool.



Bridge: From reduced spaces to anomaly detection

Dimensionality reduction	Dimensionality reduction not only helps understand structure but also exposes what doesn't fit.
Key insight	• Normal data follows consistent structure → captured by low-dimensional representation. • Anomalies break that structure → appear isolated or poorly reconstructed.
Next	Dedicated algorithms like Isolation Forest and One-Class SVM make this principle explicit

Bridge: from reduced spaces to anomaly detection





5a. Classical Anomaly Detection

Anomaly detection identifies rare, high-impact events

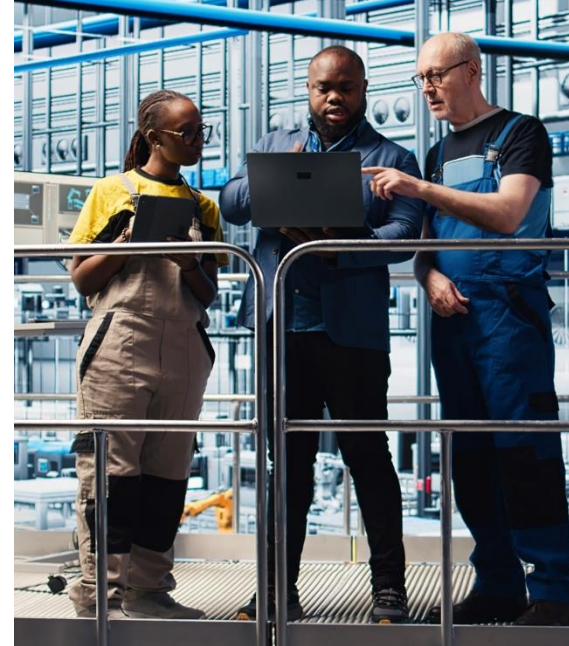
Goal:

Detect data points that deviate significantly from normal patterns



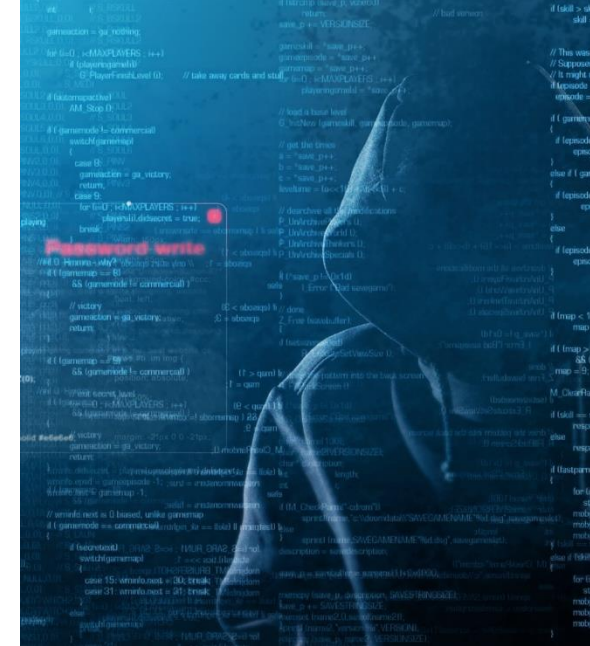
Example 1

Fraudulent transactions in finance



Example 2

Faulty sensor readings in manufacturing



Example 3

Unusual network activity in cybersecurity

Three main strategies for detecting anomalies

Approach	Concept	Intuition
Isolation-based	Separate anomalies quickly via random splits	Outliers are easy to isolate
Boundary-based	Learn a frontier around normal data	Anomalies fall outside the boundary
Reconstruction-based	Measure how well data can be rebuilt	Poor reconstruction → anomaly

Isolation Forest isolates anomalies instead of modeling normal data



Builds random trees

Builds many **random trees** that split features recursively

Key idea

Outliers are easier to isolate. →
Have **shorter path lengths** in the trees

Anomaly score

Average path length across trees
(shorter = more anomalous)

Strengths

Fast, scalable, effective in high dimensions

Figure: Forest of random splits showing shallow isolation of anomalies vs. deeper paths for normal data.

One-Class SVM defines a flexible boundary around normal data

- Trains on data considered “normal”
- Learns a **soft boundary** that encloses most of the normal data.
- Points outside → **anomalies**.

Pros

Flexible, works in complex feature spaces

Cons

Sensitive to hyperparameters, slower on large datasets

Figure: Two-dimensional feature space with One-Class SVM circular boundary; anomalies fall outside.

Comparing classical anomaly detection methods

Criterion	Isolation Forest	One-Class SVM
Concept	Randomly isolates outliers	Learns boundary around normal data
Scalability	Excellent (fast, parallel)	Moderate (depends on kernel)
Interpretability	High (path length)	Moderate (opaque boundaries)
Tuning needs	Low	High (nu, gamma, kernel type)
Best for	Large tabular data	Smaller, nonlinear feature spaces



5b. Representation-Based Anomaly Detection (Advanced Preview)

Bridge: from classical boundaries to learned representations

- 
- Classical models rely on **distance** or **boundary rules** in raw feature space.
 - Modern deep learning models learn **latent representations** of “normality.”
 - These capture richer, nonlinear relationships before evaluating anomalies.
 - Shift from **geometry-based** to **representation-based** detection.

Autoencoders reconstruct normal data and expose anomalies

Idea

Train a neural network to reproduce its input

Training

- The encoder compresses input → latent space.
- The decoder reconstructs it → output.

Anomaly rule

- Normal samples → low reconstruction error.
- Anomalies → high reconstruction error.

Strengths

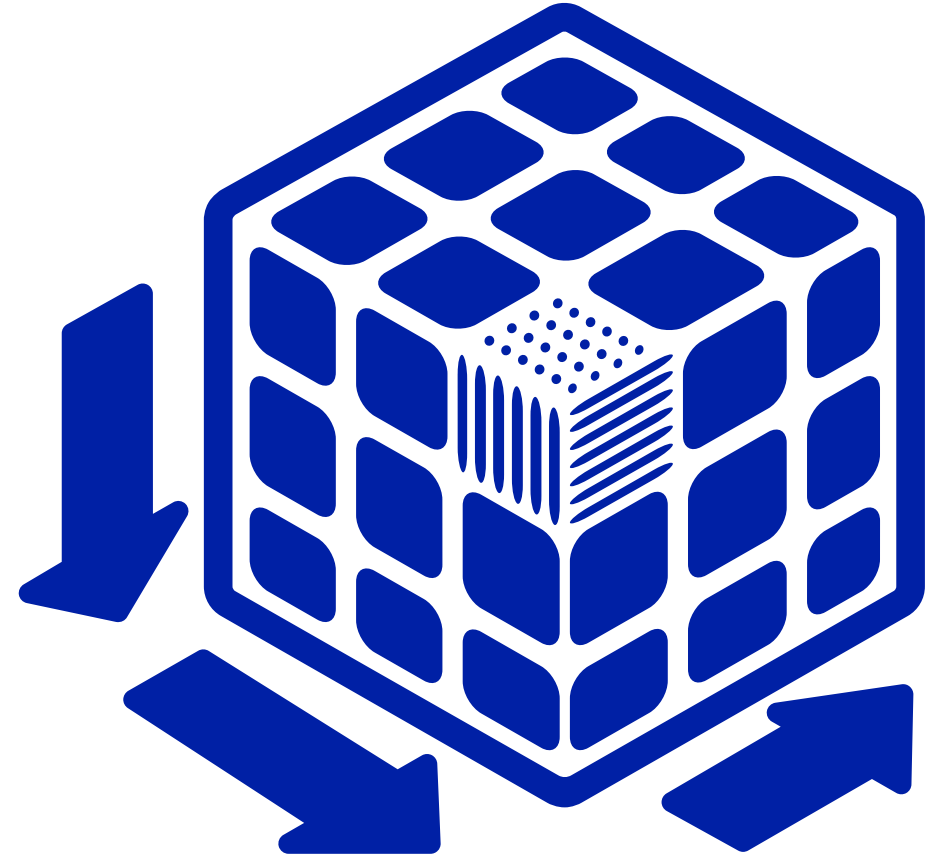
Works well on sensor data, transactions, or images

Classical vs. representation-based detection

Aspect	Classical (IF, SVM)	Representation-Based (AE, SVDD)
Feature space	Original data space	Learned latent representation
Model type	Tree / kernel	Neural network
Nonlinearity	Limited	High
Data size need	Moderate	Large
Interpretability	High	Low
Use case	Tabular / simple	Images, sensors, embeddings

Takeaway: Understanding and evolution

- **Classical methods:**
Fast, interpretable, and broadly applicable
- **Representation-based methods:**
Powerful for complex, high-dimensional data
- Both aim to model what's *normal* to detect what's *not*.
- This progression mirrors the broader trend in AI (systems to learned representations.)





6. Evaluation in Unsupervised Learning

Evaluating unsupervised models requires evidence, not accuracy



Supervised learning

In supervised learning, metrics like accuracy or RMSE are clear.

Unsupervised learning

In unsupervised learning, there are **no ground truth labels**.

The central challenge

How do we know if our discovered clusters or anomalies are meaningful?

Evaluation

Evaluation depends on **pattern coherence, visualization, Stability/robustness checks and domain validation**.

Do you have labels?

When labels exist — use quantitative validation. But beware.

- **Silhouette Score:** Measures how well points fit within their cluster vs. others. Good for checking internal consistency.
- **Adjusted Rand Index (ARI):** Compares predicted clusters to known classes. Useful only when some labeled benchmark exists. But then... are you sure you need to be doing unsupervised learning?

When no labels exist — rely on visualization and expert judgment.

- **Visualization:** Use PCA, t-SNE, or UMAP to project results into 2D/3D. Check if clusters or anomalies look plausible.
- **Domain validation:** Are discovered groups meaningful to experts? Do anomalies align with real unusual cases (e.g., known fraud, failed sensors)?

Practical takeaway: Evaluate structure, not prediction

- There is **no universal score** for unsupervised learning.
- Evaluate through **three complementary lenses**:

- ① **Structure:** Does the model reveal coherent, stable patterns?
- ② **Interpretation:** Do results make sense to experts?
- ③ **Impact:** Does it support real-world decision-making or risk reduction?

